

Daily build and feature development in large distributed projects

Even-André Karlsson

Q-Labs
Ideon
S-22370 Lund
Sweden
+46-46-2863033
even-andre.karlsson@q-labs.se

Lars-Göran Andersson

Ericsson Radio Systems AB
S-164 80 Stockholm
Sweden
+46 8 585 31768
lars-goran.andersson
@era.ericsson.se

Per Leion

Ericsson Radio Systems AB
Linköping
Sweden
+46 13 287300
per.leion@era.ericsson.se

ABSTRACT

Daily build is a software development paradigm that originated in the PC industry to get control of the development process, while still allowing the focus on end user requirements and code. The PC industry used daily build to avoid chaos in increasingly larger applications in an environment without a strong development process. Ericsson Radio Systems has chosen to implement daily build to increase the focus on end user requirements and code, but from a different starting point with a traditionally strong development process. In this article we discuss our experiences with daily build and feature oriented development in this context. We also relate our experience to the concept of extreme programming, arguing that our ideas can help extend the applicability of extreme programming beyond small co-located projects.

Keywords

Daily build, Incremental development, Feature teams, Integrated Product Teams, Extreme programming

1 Introduction

Ericsson is the world leading supplier of Mobile Communication Infrastructure. The World Class Provisioning (WCP) improvement initiative discussed in this article covers the GSM business area BMOG (Switches and base stations). Development is done in more than 15 countries and by more than 5000 developers. There are several large stand alone applications developed by rather independent product units and most products are developed over several geographically distributed sites. The development environment ranges from the traditional proprietary AXE 10 environment to a Corba/Java/UML environment for newer applications.

The WCP (World Class Provisioning) initiative was started in 1998 to secure and strengthen Ericsson's position as the world leading supplier. The goals of WCP are by 2001 to achieve the following improvements:

- 10 x Quality
- 4 x Productivity

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, to republish, to post on servers, or to redistribute to lists, requires prior specific permission and/or a fee.
ICSE 2000, Limerick, Ireland

© ACM 23000 1-58113-206-9/00/06 ...\$5.00

- 1/2 x Lead-time

To achieve these goals 10 guiding principles were chosen based on BMOG's current situation and benchmarking with other telecom and computer industry:

- Daily build and incremental development
- Customer and supplier involvement
- Cross-functional teams with sustained responsibility
- Reward for results
- Learning organization
- Instant plug and play
- Management commitment to quality
- Unbundled and open system architecture
- World class working environment
- Predevelopment

In this article we will mainly focus on the daily build and incremental development guiding principle, and the changes that they require, but in section 8 we discuss some synergies with the other guiding principles.

The WCP initiative is organized as a matrix, with persons at each product unit responsible for the WCP goals and guiding principles across their product units. Their role is to ensure that the product unit achieves the WCP goals by making efficient use of the guiding principles. The third author is the WCP responsible for the Base Station Controller node. On the other dimension there are guiding principle responsables supporting the guiding principle across all product units. The two first authors of this article are responsible for daily build and incremental development. We are responsible for promoting and supporting the product units to use daily build and incremental development.

Daily build is a software development paradigm that originated in the PC industry to get control of the development process, while still allowing the focus on end user requirements and code. A prime example of this is Microsoft [2] where a buddy pair of designer and tester are responsible for implementing a customer feature. The designer will check-out and update all modules which are needed for the feature in parallel with other designers responsible for other features. The daily build is used to allow designers to fast check the update into the main code line to avoid large merge conflicts from parallel check-outs. The designer checking in his changes first is the lucky one,

as it is up to the later designer to take care of any merge conflicts.

The PC industry used daily build to avoid chaos in increasingly larger applications in an environment without a strong development process. Ericsson Radio Systems has chosen to implement daily build to increase the focus on end user requirements and code, but from a different starting point with a traditionally strong development process originating from an environment with stable requirements from a slow regulated market. In this process the control of the code has been through detailed design documents, and coding has been done by designers implementing one module each based on design impacts from different features. Thus the code control mechanism in this process has been the thorough detailed design. Daily build is a orthogonal control mechanism which to some extent is in conflict with designers responsible for separate modules. In this article we discuss our experiences with daily build in this environment.

The guiding principle also includes incremental development. It is therefor important to clearly define what we mean as the difference between these two concepts.

Incremental development is the delivery of the functionality in increments with a well defined subset of the functionality (set of features) in each increment. These increments are planned on the project level. The frequency of increments can be rather different, down to one week for smaller projects.

Daily build is the team/individual controlled delivery of code to a main codeline which is built and tested daily. Any functionality of the delivered code is completely controlled and used by the team/individual. The daily test only covers the functionality that existed in the system at the last stable baseline.

As we see from this the main differences between daily build and incremental development are:

- Who controls and defines the deliveries
- The frequency
- The functional level of the deliveries

Daily build can be divided into three main components:

- Build
- Test
- Design

The rest of this article is organized as follows:

- Section 2 discuss the build component of daily build
- Section 3 discuss the test component
- Section 4 discuss the impacts on design
- In section 5 we describe the impacts of daily build in a distributed development environment.
- Section 6 describes the measurements for degree of implementation
- Early experiences are discussed in section 7.
- In section 8 we discuss the synergies with some of the

other guiding principles.

- Section 9 contains a discussion how the principles of incremental development and feature teams impact project planning.
- A comparison to the concept of Extreme programming is provided in section 10.
- Section 11 provides a general roadmap for how to implement daily build
- Section 12 concludes the article.

For commercial reasons, we do not disclose any facts or measurements from real projects, but all information discussed in this article has been implemented in several large multi-site projects over the last 2 years.

2 Build

One of the components of daily build is the ability to build the system on a daily basis. This means that:

- Designers must know where to put their code.
- The code must be consistent, i.e. interfaces (signal descriptions) must match bodies (modules) etc.
- We must be able to compile and load the code and any related data in much less than 24 hours.

We will discuss the impacts of the second and third requirement on the system (the first is rather evident, and is solved by a good configuration management process and tool), but first we will discuss what we mean by the system and hardware in this context. BMOG is producing all components of a GSM network out to the mobile phone, i.e. the switch (either traditional circuit switched AXE 10 or packet switched GPRS/UMTS), base station controllers and base stations, as well as the Management System. We could regard the system to be built as a complete GSM system, loaded into a GSM network. The system could also be each node in the GSM network, e.g. a base station controller. These nodes have standardized interfaces, and are usually developed by separate product units. There are usually standardized products for testing the protocols between the nodes. This is the level of system where we have focused the daily build activity currently, as this is the level where software is physically built and loaded. Ideally we should envision a daily build on a complete GSM network. We could also go into each of the nodes and look at separate subsystems as the system to be built. In this case the hardware has to be a simulator able to simulate the interfaces to the other subsystems. Doing a daily build on a lower level than system means that we loose many of the advantages, i.e. consistency check on interfaces will be local.

Consistent code means that all impacts for a specific part of an end user functionality needs to be in place at the same time, i.e. if two modules interact to do something the implementations of this interaction must be checked in at the same time. Take as an example a functionality that needs a new parameter in a function (signal). Both the caller and the called module have to implement the additional parameter at the same time. To achieve this efficiently both impacts should be implemented by the same individual/team, otherwise the amount of communications between teams to

coordinate what to check in when will be too high.

Loading the software into the hardware is not a trivial task in a telecom system for several reasons. First there exists a lot of data in the system, e.g. subscriber data, cell layout, etc. which also needs to be loaded. There are three alternatives for this loading: either a complete reload with code and data, or a so called function change where only the changed modules are loaded together with a data change information (DCI) providing a mapping from the old data structures to the new. The DCI can be done in two ways, either complete from the previous version or increment, or a partial from the last build. The partial one will be fastest but puts an extra burden on the designer as they have to provide intermediate DCI's, as well as the final one which the customer needs to upgrade the software during operation. The full DCI is slower as more modules must be loaded, but is better from an end user test perspective as we test the final DCI all the time.

The second problem with loading is the amount of sourced software in particular in the Management System which makes the loading of a system a rather complex task which needs a lot of manual intervention. This must now be automatized, but this is also for the better for the real installation, as they get an easier job.

Generally on the build part we have encountered several technical barriers, none of which have been impossible to overcome, and we are able to build most systems in well under 24 hours, even if some large legacy systems still need up to a couple of days for a complete build from scratch. Being able to build the system fast and efficiently is also an advantage for the development, independent of how often the build is performed.

3 Automatic testing

Automatic testing is the second part of daily build, as it is of little use to be able to build a system on a daily basis if we can't ensure that it has a certain level of quality. The idea of the test is to ensure that nothing of the existing functionality in the system is broken, i.e. a regression test. This test has to be automatic as the amount of effort needed to manually do the tests every day is prohibitive. There are two alternatives when it comes to what the test includes:

- The functionality level of the last increment.
- The functionality level of the last successful build.

The last alternative makes it possible for teams to include test cases to safeguard their functionality in the test, thus as soon as they have done their own test of the last build to ensure that it has the required functionality, passed test cases can be included in the automatic test. These test cases will ensure that any changes impacting their new functionality will be detected.

As discussed for build, the tests have to be on the level of the build, i.e. network, node or subsystem.

If the build fails, either during build or test, we are quite sure that it can be traced back to the last check-in, and those who have done this check-in should be the ones most natural to fix the problem. In any case we still have the last successful build.

The teams will get the tested software back and can check if their newly implemented functionality is working.

Since breaking the build is a rather serious matter, there should be a possibility for the teams to do part of the build and test steps in a private environment before submitting it to the main code line and a system build. How much of this private activity which is cost effective to perform has to be judged for each system, as it is not reasonable to do the same things twice. We recommend a scheme where each team does a code review, compile, load and some form of regression test before submitting the code. The full regression test of old functionality is done centrally. If we set the seriousness of breaking the build to high, we will get a lot of double work, if we set it too low we don't get enough progress in the build.

Usually there is a function/system test activity of the previous increment going on in parallel with the daily build activity of the current increment. We have two alternatives here:

- We can freeze the code line at the delivery of the increment, letting the function/system test proceed in a stable environment.
- We can let function/system test use the same code line as the daily build.

Using a frozen code line has the disadvantage that we must maintain two branches, i.e. serious errors must be corrected both for the frozen code line and the daily build code line. Using the same code line will mean that function/system test will execute in less stable environment, and we can never be sure if a fault comes from something delivered in the last increment or last build, similarly we can experience that test cases which passed earlier now fail because of code in the latest build which was not detected by the automatic test. However if we can include all passed test cases in the automatic test run daily we can ensure some quality here. Even if the function/system test is executed in a more unstable environment we have the advantage that faults are detected earlier.

If the amount of test cases becomes too big to be run every day, we can make a scheme where we alternate test cases so that all test cases are run at least once a week. If we have several hardware platforms we can run the test cases alternatively on different platforms to get a full coverage.

Usually there has been rather limited incentive for automatic testing, as most test cases are run only once, and the amount of regression test has been limited. In a daily build environment this is changing radically and automatic testing is a prerequisite. Automatic testing is as fast build a technical barrier, however most of the software, e.g. protocol testers and emulators are in place to perform automate test cases, the challenge is to start writing tests so they can be automatically executed. Automatic testing is also an improvement which can be of use even with less frequent builds.

4 Design impact of daily build

Up to now we have discussed the technical requirements on

daily build, i.e. fast build and automatic testing. However to have any use of daily build, design has to deliver code with some frequency, otherwise there is nothing to build. This code also has to be consistent so that it can be built. Note that there is no requirement on any functional content, as the new code is not executed by the automatic tests, but for the team to use.

There are two main connected impacts on design of daily build, the organization of responsibility and the development process.

Generally there are two ways of organizing design, either based on end user functionality where one team is responsible for a customer feature, or based on system modules where a team/individual is responsible for one or several modules and implements impacts from all features in these modules. With customer feature responsibility the team will implement changes in all modules which are necessary for the end user feature to work, possibly in parallel with other teams implementing impacts from other features in some of the same modules. These two ways of organizing design are not mutually exclusive, as we can have an organization with predominantly customer feature responsibility, but where some central modules are the responsibility of a separate team. In this case the module team has to coordinate when to implement the changes with all feature teams. Daily build can only be fully implemented in an organization with predominantly customer feature design responsibility. Note that this responsibility has to extend to the system level we want to implement daily build, e.g. network, node or subsystem. The reasons why feature responsibility is a prerequisite for taking advantage of daily build is the amount of coordination/planning needed between module responsables to deliver consistent parts of the modules which can be built if module design responsibility is used, i.e. every module designer has potential several features impacting his module, where each feature is also impacting many other modules. In a feature team these coordinations are all handled within the team. There are however the conflict with concurrent updates of the module by different feature teams, and here frequent build and automatic testing is a prerequisite for implementing feature teams, to ensure that what is checked in has the appropriate quality, i.e. has been appropriately merged with what was previously checked in.

When implementing feature team responsibility there are generally two approaches to handling several impacts on the same module, either opportunistic or planned. Opportunistic handling means that anyone can check out a module and work on it in parallel with anyone else. The first to check in their change do not have to do any merging, whereas all later have to merge with all who have worked in parallel and checked in before. The opportunistic handling provides a big incentive for making small changes to the modules and checking them in frequently, thus serving as the engine of daily build. The planned handling means that each feature team has a time window when they work alone with the module. This time window can be planned centrally or decentralized by the involved feature teams.

Note that even if we have several feature teams doing the work in the same modules does not mean that we need to give up the idea of a module responsible, on the contrary this role is getting even more important in a feature team organization. The role is changing from someone who does all the work in a module, to someone who is guiding, checking and approving solutions and implementations proposed by each feature team. Of course the module responsible should participate in the feature team which has the biggest or most complex impact on the module.

The development process is also impacted by daily build, as if we use a traditional waterfall development model there is no code to be build until very late in the project, thus we lose most of the advantages of daily build. To get full advantage of daily build we have to split the development into smaller iterations, where each iteration is taken to executable code. This means that we can adapt the amount of design documentation to the iterative way of developing software. One interesting observation when looking at design documentation is that it has two purposes: one is to support maintenance and enhancements of the system, the other is to serve as input to the next stage of the development process. For the first purpose we usually need less documentation than for the second, as in the maintenance case we can have the design information on an overview level and look for additional details in the code. On the contrary for input to later stages of development where we don't have the code, thus there is a tendency to put additional details in the high level design documentation to help the later stages. Working more iteratively on all levels of documentation at the same time will allow us to put the information directly where it belongs. This of course requires that we have a design organization where a team is responsible for the end user functionality, as otherwise we have a handover of information from many customer features to the module responsible, and this design information has to be more detailed than the information necessary for maintenance as the module designer is going to implement the impacts in the modules only from that information, and has limited time to understand each individual feature or communicate with different feature designers. It is thus natural that the development process is adapted if we use feature teams and daily build, with a higher focus on code, starting coding of parts that don't need thorough design early interleaved with design and possibly analysis of more complex parts, where information is put directly into the level of documentation where it belongs based on maintenance needs. This iterative approach should not be driven too far, i.e. the team should make a plan for how the feature should be developed in different iterations, but this is a local plan to each team, maybe coordinated with other team for access to common modules. Note that the daily building will enforce that there is such a plan, otherwise the code developed will be inconsistent and break the build.

There are some approaches to daily build where a feature team has a private branch of the code during the complete development of a feature, and only checks in at the end. This is not recommended as it usually leads to major merging

problems when several features are checked in towards the end. This could be solved with a detailed planning of the impacts of different modules avoiding the merge. This is however rather difficult as this planning has to be on a central level, putting a high strain on the planner when there are changes in requirements or wrong effort estimates.

Another consequence from parallel design is the need to be able to merge different work products easily. This is usually supported at the syntactic level of ascii files (e.g. code). For other type of documents, e.g. word processors and design tools the merging capabilities needs to be an important criteria when choosing the tool. Some tools, e.g. Microsoft Word actually have quite good support for merging parallel work.

Test is also impacted by daily build in that traditionally there has been several different test phases, e.g. basic, simulated, function and system test. The practice has been to include as much as possible in the earlier test phases, as it is more expensive to discover an error in a later test phase., and the feedback is longer. The disadvantage with all these test phases is a lot of overlap, i.e. the same functionality tested several times and a long lead-time. With daily build and automatic testing this assumption is not necessarily valid any more, as we can do any sort of tests after the daily build. Thus a feature team can and should test the functionality where it is most efficient, and only once.

5 Impacts of daily build

We divide these impacts into two groups, one concerning the advantages from frequent build and automatic test of the system, the other from organizing design in features teams to deliver frequently. We also discuss some pitfalls when reorganizing design to take advantage of daily build.

Frequent builds and automatic testing

The major advantages of being able to build and test the system daily are:

- **Stability** We always have an executable system, even if the current build fails we can go back to the last stable one.
- **Test** Teams can always integrate their code into a running system and test it on a system level.
- **Follow up** We see progress very clearly, i.e. not only document statuses but real executable code. If a team has not delivered any code for a long time, we know that it is something which should be investigated.
- **Feedback** We can get early and continuous quality feedback. Thus if a team is delivering substandard code it will be immediately discovered and can be adjusted before the damage is too big.
- **Early fault detection** Faults are detected early after they are introduced. This does not only pertain to local faults but also global system faults and interference with old functionality. These types of faults are usually discovered later without daily build.
- **Isolated errors** It is easy to isolate errors, as they in most cases come from the new code in the current build, and the teams delivering this code should be able to fix the

errors.

- **Parallel system test** System test can proceed in parallel with development, thus reducing the lead-time for system test at the end.
- **One code line** When using only one main code line for development and system test, the need for parallel code updates disappears.

Feature teams

The major advantages of using feature teams as described in section 4 are:

- **Motivation** The feature team has a focus on end user functionality and code. They have the full responsibility for implementing the feature, and use the existing modules as means to achieve this functionality.
- **Change management** Changes can be absorbed more easily by the feature team, i.e. both external requirements changes and changes in the design can be handled within the team without impacting other teams. As most changes are pertaining to a specific feature this is one of the major advantages of feature teams. With module responsibility the feature changes have to be analyzed on a central level and distributed to each affected module. If there are changes in the design of one module for a feature and this impacts the work load for the module responsible other features may need to be rescheduled leading to more module responsibilities being impacted. This is one of the effect we see when projects are doing frequent (down to weekly) centrally planned increments with module responsibility, the plans get very volatile to both external and internal changes, and the work load on the architects and planners is very high.
- **Coordination** towards associated projects. Usually a requirement towards an associated system is because of a feature. If the interaction to the associated system is through a module which have impacts from many features, the module responsible will not be able to fully understand the reason for each requirement, and can not efficiently communicate with the associated system. This leads to an inefficient coordination if we don't use feature teams, which are responsible for both the feature and the impacts in the interface module and can make efficient trade-offs with the associated system.
- **Interface coordination** Usually changes in interfaces between modules are because of new features which need extended interaction. When we use feature teams the interfaces are handled within the team, when using module responsibility the interfaces have to be settled before each module is implemented to allow parallel work. This is usually an impossible task, leading to time consuming coordination, rework and integration problems.

There are also pitfalls when using feature teams. The major being the potential degeneration of each module and interfaces. This is a real problem which we propose to solve with a module coordinator which is approving and checking what each feature team implements in the module.

The focus on early code can lead to an ill structured design of a feature where later iterations will have to redo coding

done in earlier iterations. To avoid this we need a certain level of analysis and planning for the development of the feature in iterations by the feature team.

Another constraint is having a distributed development organization where different sites are cooperating on a project. This means that either we have to have people from different sites in the same team or moving modules between different sites. In any case this increases the demands on travels and computer supported cooperative work tools, i.e. efficient configuration management and teleconferencing equipment.

6 Measurements

In the WCP program we have the three overall goals on quality, productivity and lead-time which are followed up by measurements. But we also want to monitor the degree of implementation of daily build, both for individual projects but also aggregated over product units. For that we have designed the following measure:

Degree of implementation of daily build is measured by the number of days with successful builds during the design lead-time divided by the number of days for design.

In this measurement the lead-time of design is up to each organization to define, but is usually measured from the end of feasibility (decision to start the project) to the start of function test of the last increment. Also the criteria for a successful daily build is up to each organization to define. It has three components:

- The daily build has to be above some size, i.e. there has to be some new code delivered. We recommend to use the limit of new code > 0.5 total code / number of days for design
- The daily build and test has to run successful.
- The executable system has to be distributed to the users.

Note that this measure can be used to continuously follow up the degree of implementation by only counting the days of design up to today. The reason for stopping the counting as design is completed is that daily build is a design activity, not primarily a means to speed up error correction during test, thus even if we will use daily (and possibly more frequent) builds during final testing, we don't include that in our measure.

For a product unit with several products the aggregated measure is a weighted average by size of effort over the different products.

This gives us a simple way to quantify the progress of implementing daily builds through the complete WCP program.

We have also some recommendations for projects on how to follow up the impacts daily build has on the WCP goals, i.e. quantification of the impacts discussed in section 5. This is usually a rather complex relationship which has to be adapted for each organization based on their reasons for doing daily build and their implementation.

7 Early experiences

Our current experiences with complete implementation of daily builds are not complete, meaning that we have different projects all on the way towards daily build, but none with a complete implementation as defined by our metrics.

The following is a cross sample of the different projects:

A) This project is a distributed project over 5 sites involving about 50 designers. It is developing a base station controller module from scratch. The project has implemented daily builds and automatic regression test. There is a build daemon which does the build and test every night (or even more frequent), and distributes a mail message about the result. The organization of design is however still based on design responsibility, and frequent (down to twice a week) increments are planned on the project level. This makes the project rather volatile to changes, and this level of detailed planning is probably at the limit of what can be done centrally. The design is robust against incomplete interface, as each module will accept all calls, but just discard with an error message any call that is not implemented, thus the daily build and automatic test can proceed without coordinated interfaces as long as they are not used in the test.

B) This project is making enhancements to an existing node in the network. It is organized at one site, and has used feature teams for a long time. Daily build and automatic test is not implemented, but the system is built whenever the teams check in anything. This project has focused on the impacts of feature teams on the development process, where they have discovered that there can be a rather dramatic reduction in design information by using feature teams and parallel work on design and code. One example of a function description being reduced from 200 to 9 pages was reported.

C) This project is a large multi node project involving more than 500 developers. In this project we have introduced feature teams on the network level. The project is planned with four increments, two of which are pre-releases to early customers. One of the nodes have changed from module oriented to feature oriented development, with the modification that some key modules are handled by separate teams. A daily build environment is introduced in this node. Some early experiences are that we have been able to reduce the total lead-time of some critical features with 40% by iterations inside the feature teams.

8 Synergies with other guiding principles

Even if the changes described in the earlier sections originated from the daily build and incremental development guiding principle, they have important synergies with several other guiding principles.

Cross functional teams with sustained responsibility

The feature is the natural unit of functionality that we develop and deliver to our customers, thus it is the ideal task for a team. The feature team is then responsible for getting the feature to the customer within a given time, quality and budget. A feature team needs to be cross functional as it needs to cover all phases of the development process from customer contact to system test, as well as all areas of the system which is impacted by the feature.

Incremental development

Incremental development is the project level planning of features to increments based on customer prioritizing. On the project level we must ensure that

- each feature team has the right resources,
- they make realistic plans, and
- follow their plans

This way of planning gives much more stable plans on the project level of the following two reasons:

- Most changes are directly related to one feature, and will only impact one feature team.
- A feature team can easily adjust to smaller changes in plans, e.g. opening a new module for a minor impact, changing the effort from one module to another, etc. without impacting anyone else. Also larger impacts would only mean that the feature will be rescheduled for delivery in a later increment.

In the traditional product organization these type of changes have unnecessary large consequences as they spread like rings in the water because features are interconnected through the module implementers.

We should also on the project level planning ensure that features are grouped in increments to allow for the best possible scheduling of impacts to critical modules. Some examples:

- If we have many impacts from different features to some modules we should try to schedule the features evenly over the different increments to have a even work-load on the modules.
- If we have one feature which (re)builds the infrastructure in one module, this should be done before other features impacting this module.

Customer and supplier involvement (subcontract management)

Customer involvement is very natural with feature teams (compared to our traditional way of organizing where development is organized around products (modules)). The feature team can have continuous contact with the customer, and show him early versions of the feature working in the system (through daily build). Also the prioritizing of features in increments can allow us to be more flexible with the customer, but on a higher level, e.g. if the customer decides that a new feature is very important it can replace another in a late increment.

Another important aspect of feature teams is in relationship to supplier involvement (subcontract management). Here most requests on a subcontractor come because of the new features. It is then very natural for the feature team to keep close contact with the subcontractor to ensure that they find the optimal solution for their feature. This contact is much more difficult when development is organized around products (modules), and the responsables for implementing the two interfacing modules have several different features to think of, and can not easily make any global changes to achieve a better solution for the feature. A related issue here

is the interface coordination between different parts of the product if we use product responsibility in development. This will to a large extent disappear if we use feature teams, as most changes to interfaces are due to the feature, and the feature team is responsible for both the interfaces and the implementations.

Reward for results

The goal of a feature team is rather clear and independent, i.e. deliver a feature with required quality that satisfy the customer requirements. It is then easy to reward the team for accomplishing this goal.

Learning organization

Working in feature teams with fast feedback (daily build) will allow the organization to learn much quicker.

Part of World Class Working Environment (ClearCase)

The configuration management tool (in our case ClearCase) is a support infrastructure for daily build, and thus feature teams. It helps to keep track of versions for the team itself, and also to consistently select versions for build (Make).

9 Impacts of incremental development and daily build on project planning

This section discusses how incremental development and feature teams impact the different decision points (tollgates) and interfaces between product and project management.

Baseline situation

The interface between product and project management and the use of the tollgates has not been optimal. The initial scope of the projects has been far too big, and the early tollgates have been used mainly to limit this scope. Project management tried to limit the scope as fast as possible to get the project under control, and be able to start realistic planning, as planning with a unrealistic scope was not possible, and product management was afraid of excluding any features before they really had to as there was little chance for getting the features in again. Also previous projects have experienced overruns in cost and lead-time which there has been no possibility to adjust by cutting scope as all features have been developed in parallel. Thus there is a situation where pre TG2 (decision to start execution of project) scope is reduced as much as possible, and after TG2 we could just hope that the project will meet their cost and time goals with the decided scope, as there was little flexibility in adjusting the scope.

The new model for projects

In the current environment we have a more flexible set of features throughout the project based on customer priority, stability of features and project resources.

We describe an example of how such a project divided into four increments would look.

At TG0 the overall scope of the project is set in terms of

- end date,
- total resources, and
- direction (only new features, restructuring, system improvements, etc.)

- All new features are grouped into four categories:
- **Group 1** Very important and stable feature, needing 25-35% of total resources (candidates for first increment)
 - **Group 2** Important and relatively stable feature, needing 40-50% of total resources (candidates for second and third increments)
 - **Group 3** Other important features, needing 25% of total resources (candidates for last increment)
 - **Group 4** Other interesting features, up to 50-100% of total resources.

Note that these four categories totals to 150%-200% of project resources, but we only expect to complete the three first categories. Features in group 1 is put on a “fast track” through prestudy (specifying requirements) and feasibility (implementation proposal and cost estimates) for early start of execution in the first increment. Features in group 2 are driven through prestudy in the usual speed. Features in group 3 are given less priority, and only prestudy for complex ones are started. Features in group 4 are in reserve, and no resources are used on them from the prestudy project.

Project management can now start preliminary planning of resources based on this grouping. This does not mean that the plans will not be changed - they indeed will, but at least there is a baseline to change.

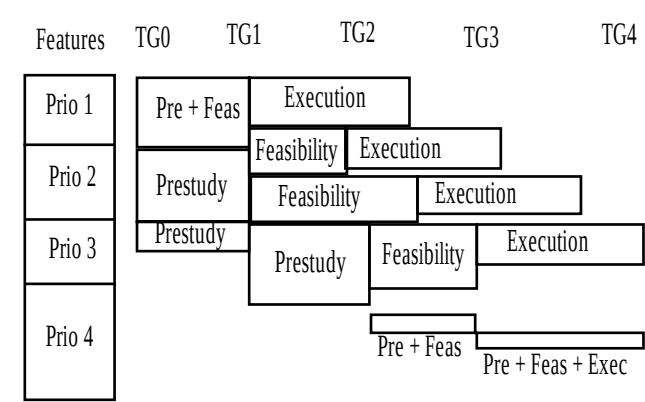
At TG1 the overall scope of the project is adjusted. A decision can be taken to start execution (as soon as possible) for features in group 1. Based on the output of prestudy of features in group 2, a suitable set for increment 2 should be determined, and feasibility for this started. The scope for prestudy for increment 3 is adjusted, and any features not prestudied are started. Note that at any point features from group 4 can replace features in the other groups. The tollgates are used to do this regrouping.

At TG2 the execution content of increment 2 and 3 are decided and execution of increment 2 started. Content for feasibility for increment 4 is decided and started. Any additional features popping up are started on a fast track through prestudy and feasibility.

At TG3 the execution content of increment 4 and the final scope of the project is decided.

The following figure gives an idea about how this could look. Here we could assume 3 months between each TG, giving a lead-time TG0-TG4 of 12 months. The list of features in each priority group would change during the project, and a new feature can come into the project as late as TG3 if it can be run through prestudy, feasibility and execution in 3

months.



As we see here there will be a continuous adjustment of features during the project. The tollgates will mainly be used to decide on the overall scope of the project, i.e. functionality versus lead-time and resources, whereas individual features can be adjusted at any time.

Why feature teams?

This will be a rather complex project environment, where the main project steering mechanism is the current list of features, their status and estimated completion cost. Delegating each feature to a team which has the complete responsibility to develop this feature will then make the project follow up and replanning much easier. Removing one feature will only affect one team, and it is easier for different teams to work out of phase.

Feature teams are also an advantage for product management, as they can then directly communicate with the feature team, and also see early versions of the feature.

Trying this way of running a project in a traditional organization with module responsibility would most probably fail, as it is impossible to predict the work for each module responsible during the different phases of the project, or this will drastically reduce the flexibility of replacing features.

Allocating a feature to a feature team at one site will also allow us more flexibility in when we start development of each feature, i.e. when we have resources (with the necessary competence) at one site we can start a feature there, and we avoid all the coordination with resources on other sites which previously needed to work synchronously. The feature team will of course need support and guidance from other sites for products (modules) they are not familiar with, but this is less than having the other site to do the work. Note that the feature team has to have the core competence for developing the feature.

Note that our flexibility is not complete, as our resources are not completely interchangeable even if we use feature teams, thus if we want to replace one feature with another, and the team intended for the first feature does not have the competence for developing the other feature we need to reallocate people between teams. Thus product management does not have complete freedom.

Another advantage of using feature teams is that we avoid the tendency of “delay creep”, i.e. a project organization where module responsables are implementing all impacts on a module it is very “easy” to accept a “small” change request, as it is just minor additions for several people, similarly bad feature estimates are spread over several persons, but all these small increases add up to large “hidden” delays and cost overruns. In a feature team environment these problems will still be there, but they will be more “isolated” and visible, i.e. this feature will be delayed, thus we can not start on the next as planned.

10 Relation to Extreme programming

Extreme programming (XP) was conceived and developed to address the specific needs of software development conducted in small teams in the face of vague and changing requirements [1]. It is a light-weight methodology challenging many conventional assumptions. In this section we discuss how our approach relates to XP, and propose some extensions that could be useful for it to scale up.

The **planning game** is well covered already in our practices. Currently we do the planning on two levels, features into different increments of a release where there is a constant discussion between product management (business) and the project (technical) on what can be included. The discussion is on the feature level. Then each team is responsible for the development of each feature.

Small releases is a practice which is not too simple, as the roll-out of new telecom software into a network is a rather complex process, but we are increasing the potential delivery frequency from 12 to 3-4 months.

The **metaphor** is a less established practice, as we still have a lot of very detailed high level documentation, and actually sometimes lacking the 10.000 feet view.

Simple design is not done well. We still tend to use far too much time up front to develop the full design on paper. However we believe that as we get more experience this will start to change.

Automatic testing is starting up. We are discussing the need for basic tests to cover all functionality, i.e. if there is an automatic user test which covers several basic test cases in different modules, why should we then have the basic test cases as well? With the feature teams we also solve the problem mentioned in Beck's book [1] on page 118, i.e. how to synchronize the function tests with design, as the feature team takes care of both development and test. However the concept of writing (basic) test cases first is not practices to any large extent yet, However user test cases are written before the code, and we are also discussing to let the test cases take over more of the functional specifications.

Refactoring is also a practice that we have not utilized in the XP way. We do restructurings of our systems, but these are usually large and traumatic experiences.

Pair programming is not used as described in XP. This will be one of the practices that we need to put in place when we want to reduce the amount of high level documentation and up front design even further than what can be achieved with

a feature team. There is an interesting combination of pair programming with module architects/coordinators, i.e. there should always be a module coordinator in the pair when complex things are done with a module. This would be a good way to ensure the consistency and competence transfer in a large system.

Collective ownership is started with the introduction of feature teams, where the feature team is impacting all modules necessary for the feature.

Continuous integration is implemented through daily build. Note that we have this mechanism on two levels, on for the feature team themselves on a private branch as well as for the complete system. The reason for keeping these two branches separate is to reduce the impacts of any errors.

40-hour week is not practiced as described in XP, however since Ericsson is an European company, there is a higher adherence to normal working hours.

On-site customer is not trivial, as Ericsson is developing products for the open market, we do however often have a product management (internal customer) representative in each feature team.

Coding standards are already quite well implemented in Ericsson. We do have a thorough code review process called desk check of all code written which enforces this. The desk check could with the introduction of pair programming be reduced.

In summary we have started to apply some of the principles of XP. Our major contribution to the XP practices might be the focus on feature teams and the introduction of the module architect role to allow the other practices to scale up.

11 General roadmap

To support the organizations implementing daily build we have provided a general road-map. This road-map covers the essential aspects of daily build, and let the organization assess where it is according to these aspects. The road-map is to be used for each project. The basic ingredients of the road-map are as follows:

Build

- CM tool in place (e.g. Multisite Clearcase)
- Daily build on team level
- Daily build on subsystem level
- Daily build on system level

Automatic test

- Basic test level
- Simulated function test
- Target test (function and system)
- Automatic notification of errors

Design

- Merge tool to support for Code (ascii)
- Merge tool to support for Design (e.g. documents, SDL, UML)
- Feature teams on subsystem level

- Feature teams on application level
- Feature teams on system level

12 Conclusion

Implementing daily build and feature teams in an organization with a strong development process, distributed development and a tradition of module responsibility is not an easy task. We have currently implemented all aspects described in this article in different projects, but we have still to see a complete implementation of the whole concept in a completed project, the first large projects approaching this degree of implementation will complete during 2001. Even

so we can already see large advantages of the elements that we have implemented, as well as future improvements, e.g. from extreme programming.

REFERENCES

1. "Extreme Programming explained, Embrace Change", Kent Beck, Addison-Wesley, 2000.
2. "Microsoft Secrets: How the worlds most powerful software company creates technology, shapes markets and manages people", Richard W. Selby and Michael A. Cusumano, Simon & Schuster, 1998